

Escalabilidade com afinidade de threads no NPAD

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Avaliar como a escalabilidade do código de Navier–Stokes muda com os tipos de afinidade de threads suportados pelo sistema operacional e pelo OpenMP, no mesmo nó de computação do NPAD usado na Tarefa 12.

2 Fundamentação

O experimento usa, sem alterações numéricas, a versão `v4_numa.c` da Tarefa 12. O programa calcula a parcela difusiva em uma malha bidimensional com um stencil de cinco pontos e dois campos em `double`. Executa 500 passos e distribui blocos contíguos de linhas com `schedule(static)`. A barreira implícita do `omp for` é necessária porque o passo seguinte lê o campo produzido no passo anterior.

A inicialização paralela escreve os dois campos antes da evolução. Pelo princípio de *first touch*, essa escrita influencia o domínio NUMA associado às páginas. Manter a afinidade entre a inicialização e o stencil favorece o acesso local à memória. A diretiva `omp simd` atua no laço interno, enquanto duas variáveis de ambiente controlam a afinidade:

- `OMP_PROC_BIND=false`: o runtime não fixa a afinidade das threads e o sistema operacional pode migrá-las;
- `OMP_PROC_BIND=close`: threads vizinhas ocupam lugares próximos;
- `OMP_PROC_BIND=spread`: o runtime distribui as threads entre os lugares disponíveis;
- `OMP_PLACES=cores|threads`: cada lugar representa um núcleo físico ou uma CPU lógica.

O experimento comparou quatro configurações: sem binding, `close+cores`, `close+threads` e `spread+threads`. A combinação `spread+cores` foi medida na Tarefa 12 e não integra os resultados deste relatório.

3 Experimento

As quatro configurações foram executadas no nó `r2n19` do NPAD, com uso exclusivo e um processo. O nó possui dois AMD EPYC 7713, 64 núcleos por soquete, duas CPUs lógicas por núcleo e oito domínios NUMA. Todas as configurações usaram o mesmo conjunto de CPUs permitido pelo Slurm, 0-63, 128-191, e os mesmos domínios de memória. A Tarefa 12 usou o nó `r2n04`, de mesma topologia. A mudança de nó não atende à exigência do enunciado; a comparação deste relatório se restringe às quatro configurações executadas em `r2n19`.

As configurações usaram o mesmo código e o mesmo binário. Apenas `OMP_PROC_BIND` e `OMP_PLACES` mudaram. O Anexo A apresenta o programa completo sem a instrumentação do PaScal e mantém a conversão da carga-alvo em tamanho de malha.

O PaScal Analyzer combinou $p = 1, 2, 4, 8, 16, 32, 64$ threads com sete cargas-alvo de 262 144 a 16 777 216 células internas, dobradas a cada nível. Para uma carga-alvo W , a malha quadrada tem lado $N = \text{round}(\sqrt{W}) + 2$, incluindo as bordas. O arredondamento pode fazer a carga efetiva diferir do alvo. Cada execução usou a condição inicial gaussiana e 500 passos.

Cada configuração teve 49 pontos, com uma execução por ponto e sem estimativa de dispersão. Os mapas usam a região *Whole Program (Region 0)*, que inclui alocação, inicialização e evolução.

4 Resultados

O mapa *Scalability* do PaScal Viewer combina crescimento de carga e de recursos. As linhas representam $p = 2, 4, 8, 16, 32, 64$ e as colunas $i2-i7$ representam cargas crescentes. Tons azuis indicam regiões relativamente favoráveis na escala normalizada do Viewer; tons vermelhos indicam perda relativa. O mapa não fornece tempo em segundos. Este relatório não quantifica tempos, speedup ou eficiência, nem apresenta análises separadas de escalabilidade forte e fraca.

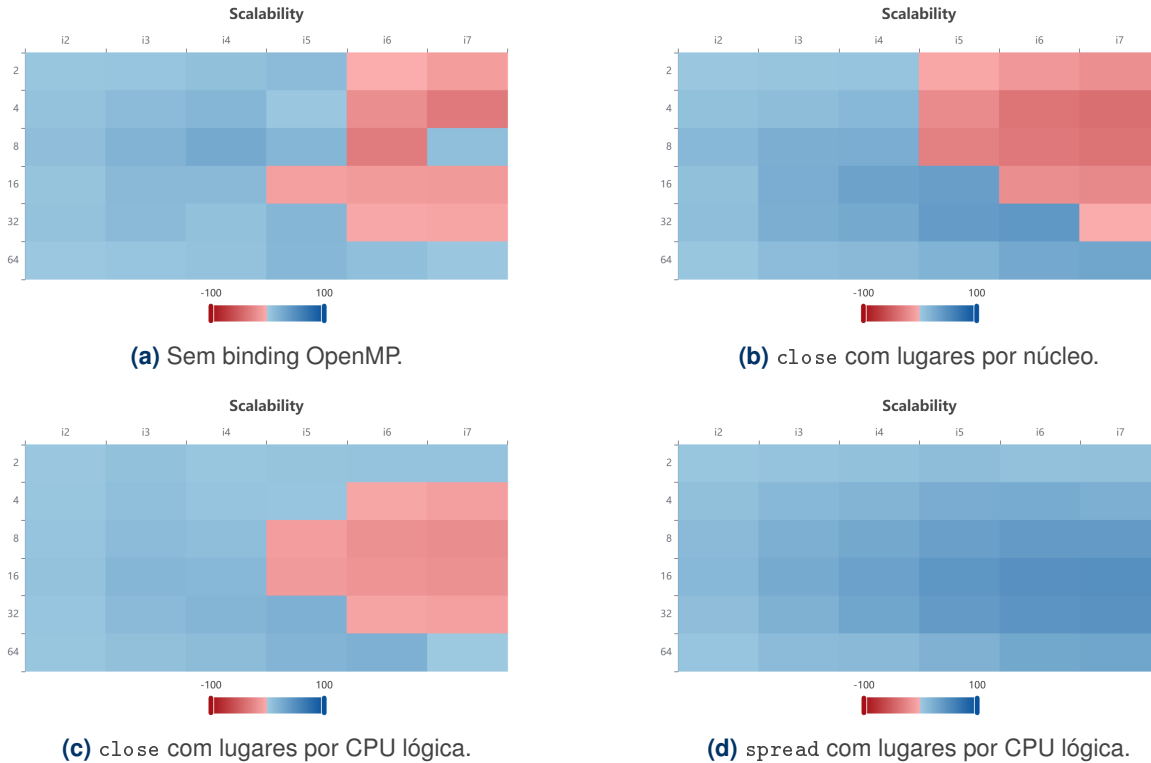


Figura 1: Escalabilidade geral do programa inteiro nas quatro configurações. As linhas indicam o número de threads e as colunas, cargas crescentes.

Sem binding, o mapa alterna regiões favoráveis e desfavoráveis nas cargas maiores. O sistema operacional pode migrar as threads nessa configuração, mas não houve medição de migrações que permita relacioná-las à irregularidade observada.

Com `close+cores`, as cargas menores aparecem em azul e a faixa favorável avança para cargas maiores quando p cresce. A afinidade mantém as threads em lugares próximos, mas o mapa não permite determinar quanto essa proximidade contribuiu para o resultado.

Com `close+threads`, a região desfavorável aparece nas cargas maiores para valores intermediários de p e desaparece em $p = 64$. A diferença em relação a `close+cores` mostra que a escolha entre lugares por núcleo e por CPU lógica afetou o resultado. Não é possível isolar a contribuição do SMT.

`spread+threads` é a única configuração com todas as células em azul. Os tons mais intensos concentram-se nas cargas médias e grandes entre 8 e 32 threads, e o mapa permanece favorável em 64 threads. Essa configuração distribui as threads entre os lugares disponíveis e apresentou o padrão mais regular no indicador geral do Viewer.

Anexo A: Código completo sem instrumentação do PaScal

O programa abaixo mantém o cálculo, a inicialização, as verificações de entrada e alocação, os cronômetros OpenMP, os diagnósticos e a saída opcional em arquivo. Foram removidos apenas os trechos específicos da instrumentação do PaScal.

```
1 #define VERSION "v4_numa"
2 #include <errno.h>
3 #include <limits.h>
4 #include <math.h>
5 #include <stdint.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include <omp.h>
10
11 static double *evolve(double *u, double *v, int n, int steps, double r,
12     int *threads);
13
14 static int integer(const char *text, int minimum, int *out) {
15     char *end;
16     errno = 0;
17     long value = strtol(text, &end, 10);
18     if (errno || end == text || *end || value < minimum || value > INT_MAX
19         )
20         return 0;
21     *out = (int)value;
22     return 1;
23 }
24
25 int main(int argc, char **argv) {
26     int n = 512, steps = 500, mode = 0, threads = 1;
27     if (argc > 5 || (argc > 1 && !integer(argv[1], 3, &n)) ||
28         (argc > 2 && !integer(argv[2], 0, &steps)) ||
29         (argc > 3 && (!integer(argv[3], 0, &mode) || mode > 2))) {
30         fprintf(stderr, "Uso: %s [carga>=3 [passos>=0 [modo=0|1|2 [campo.
31             bin]]]]\n", argv[0]);
32         return EXIT_FAILURE;
33     }
34     if (argc > 1) {
35         int side = (int)round(sqrt((double)n));
36         n = side + 2;
37     }
38     if ((size_t)n > SIZE_MAX / sizeof(double) / (size_t)n / 2) {
39         fprintf(stderr, "Malha excede o espaço de endereçamento\n");
40         return EXIT_FAILURE;
41     }
42     size_t count = (size_t)n * n;
43     double total_start = omp_get_wtime();
```

```

41 double *u = malloc(count * sizeof(*u)), *v = malloc(count * sizeof(*v)
    );
42 if (!u || !v) {
43     fprintf(stderr, "Falha de alocação para N=%d\n", n);
44     free(u);
45     free(v);
46     return EXIT_FAILURE;
47 }
48 const double nu = 0.1, dt = 0.2, dx = 1.0;
49 const double r = nu * dt / (dx * dx);
50 double init_start = omp_get_wtime();
51 #pragma omp parallel for schedule(static) default(none) shared(u,v,n,
    mode)
52 for (int i = 0; i < n; ++i) {
53     for (int j = 0; j < n; ++j) {
54         double x = i - (n-1)/2.0, y = j - (n-1)/2.0;
55         double sigma = n/10.0;
56         int boundary = i == 0 || j == 0 || i == n-1 || j == n-1;
57         size_t c = (size_t)i*n+j;
58         u[c] = mode == 2 ? 1.0 : (mode == 1 || boundary ? 0.0 :
59             0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
60         v[c] = u[c];
61     }
62 }
63 double init_seconds = omp_get_wtime() - init_start;
64 double start = omp_get_wtime();
65 double *field = evolve(u, v, n, steps, r, &threads);
66 double seconds = omp_get_wtime() - start;
67 double total_seconds = omp_get_wtime() - total_start;
68 double minimum = field[0], peak = field[0], sum = 0.0;
69 for (size_t c = 0; c < count; ++c) {
70     if (!isfinite(field[c])) {
71         fprintf(stderr, "Campo nao finito\n");
72         free(u);
73         free(v);
74         return EXIT_FAILURE;
75     }
76     minimum = fmin(minimum, field[c]);
77     peak = fmax(peak, field[c]);
78     sum += field[c];
79 }
80 printf("%s,%d,%d,%d,%d,%.9f,%.9f,%.9f,%.17g,%.17g,%.17g\n",
81     VERSION, n, steps, mode, threads, init_seconds, seconds,
82     total_seconds,
83     minimum, peak, sum);
84 int status = EXIT_SUCCESS;
85 if (argc == 5) {
86     FILE *file = fopen(argv[4], "wb");
87     if (!file) {

```

```

87         perror(argv[4]);
88         status = EXIT_FAILURE;
89     } else {
90         if (fwrite(field, sizeof(*field), count, file) != count)
91             status = EXIT_FAILURE;
92         if (fclose(file))
93             status = EXIT_FAILURE;
94         if (status)
95             fprintf(stderr, "Falha ao escrever campo\n");
96     }
97 }
98 free(u);
99 free(v);
100 return status;
101 }
102
103 static double *evolve(double *u, double *v, int n, int steps, double r,
104 int *threads) {
105     #pragma omp parallel default(none) shared(u,v,n,steps,r,threads)
106     {
107         double *a = u, *b = v;
108         #pragma omp single
109         *threads = omp_get_num_threads();
110         for (int t = 0; t < steps; ++t) {
111             #pragma omp for schedule(static)
112             for (int i = 1; i < n-1; ++i) {
113                 #pragma omp simd
114                 for (int j = 1; j < n-1; ++j) {
115                     size_t c = (size_t)i*n+j;
116                     b[c] = a[c] + r * (a[c-1]+a[c+1]+a[c-n]+a[c+n]-4*a[c])
117                     ;
118                 }
119             }
120             double *tmp = a;
121             a = b;
122             b = tmp;
123         }
124     }
125     return steps % 2 ? v : u;
126 }

```